Day 85: Word Embeddings — One-Hot Against Dense, on a Real Vocabulary

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 2 | Complete study guide

Watch: What are Word Embeddings? (IBM Technology) | Word Embedding and Word2Vec, Clearly Explained!!! (StatQuest with Josh Starmer)

Script: `day85_word_embeddings.py` · every number in this guide is copied by program from `day85_run_output.txt`, the real output of that script.

A network needs vectors, not ids. A one-hot vector is the simplest: as long as the vocabulary, all zeros but one. Multiplied by a matrix it simply picks out one row, so a dense embedding is the same operation made cheap. On Week 9's tagging task, a window tagger with a learned table of only 8 numbers per word reaches 0.8528 test accuracy, against 0.8190 for one-hot input with 20 times as many weights; a random table that is never trained reaches 0.6972. And the learned table puts words with the same part of speech next to each other: 0.854 of the 500 commonest words have a nearest neighbour with their tag, against 0.138 for a random pair.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

A **one-hot vector** for word t in a vocabulary of V words has V numbers: 1 at position t, 0 elsewhere. A **dense embedding** is row t of a table E with V rows and d columns, d much smaller than V; the table is learned with the rest of the network. One-hot times a matrix equals a row of that matrix, so the two differ in cost, in size, and in whether rows can be shared and learned.

REAL-WORLD SCENARIO

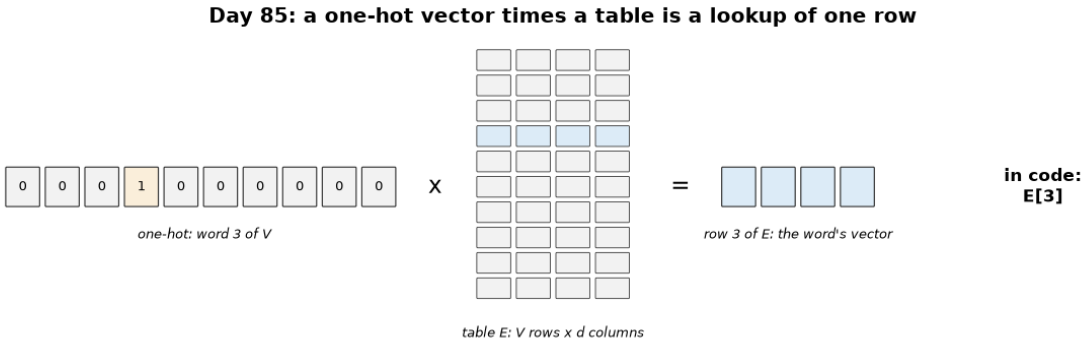
A library can label each book with a slot in a wall of 2,000 pigeonholes (one-hot), or describe it with a few numbers such as subject, reading level and length (dense): books with similar numbers are similar books.

MATHEMATICS

$$\text{one-hot}(t) \cdot E = E[t, :] \quad \text{(the t-th row)}$$

$$\text{distance}(\text{one-hot}(s), \text{one-hot}(t)) = \sqrt{2} \text{ and one-hot}(s) \cdot \text{one-hot}(t) = 0 \text{ for every pair } s \neq t$$

SYSTEM DIAGRAM



A one-hot vector times a table is a lookup of one row.

Today's part for Day 90

`day85_exercise.py` is an embedding layer, forward and backward, gradient-checked: Week 13's second part.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. A one-hot row times a matrix is a lookup

DEFINITION — start here, no prior knowledge assumed

1,000 random words of Day 62's 2,082-entry vocabulary, as one-hot rows multiplied by a 2,082 × 32 table, and as a direct lookup of rows.

REAL-WORLD SCENARIO

Finding a book by scanning all 2,000 pigeonholes, or walking straight to its number.

ACTUAL OUTPUT

```
largest |one_hot @ E - E[ids]| over 1,000 words: 0.0e+00
time for the 1,000 words: matrix product 2.860 ms, lookup 0.011 ms (the product is 266 times
    slower here; only the ratio is meaningful)
memory for the inputs: one-hot 16.7 MB, ids 4.0 kB
Actual output from this lesson's run.
```

What it means

The results are identical (0.0e+00). The matrix product does V times the work to get the same row: 266 times slower here, and the one-hot inputs take 16.7 MB against 4.0 kB for the ids. Every embedding layer is this lookup.

WHY CHOOSE THIS?

The lookup is exact and almost free.

WHY NOT THIS? (TRADEOFFS)

Nothing; the one-hot product is only a way to see what the lookup means.

2. One-hot vectors know nothing about words

DEFINITION — start here, no prior knowledge assumed

The distance and dot product between one-hot vectors of different words.

REAL-WORLD SCENARIO

Pigeonhole numbers say nothing about which books are alike.

ACTUAL OUTPUT

```
distance(word 5, word 6) = 1.4142; distance(word 5, word 700) = 1.4142 (sqrt 2 = 1.4142); dot
    products 0 and 0
Every pair of different words is equally far apart and has similarity 0: 'good' is no closer to
    'great' than to 'the'.
Actual output from this lesson's run.
```

What it means

Every pair of different words is 1.4142 apart with dot product 0. A model reading one-hot vectors must learn about each word separately; nothing it learns about 'good' carries over to 'great'.

WHY CHOOSE THIS?

No assumptions: every word starts equal.

WHY NOT THIS? (TRADEOFFS)

No sharing: similar words cannot help each other.

3. One-hot, dense and frozen inputs on real tagging

DEFINITION — start here, no prior knowledge assumed

The same tagger reads a word and its two neighbours: as three one-hot rows, as three rows of one learned table (d = 2 to 128), or as three rows of a random table that is never trained.

REAL-WORLD SCENARIO

Three ways of describing the same books to the same librarian.

MATHEMATICS

$$\text{one-hot: } h = \tanh([1(w_{-1}); 1(w_0); 1(w_{+1})]\, W_1 + b), \quad W_1: 3V \times 64$$

$$\text{dense: } h = \tanh([E[w_{-1}]; E[w_0]; E[w_{+1}]]\, W_1 + b), \quad E: V \times d \text{ shared by the three positions}$$

SPECIFICATION

Data	Day 62's split and vocabulary (2,082 entries), PAD at sentence edges.
Training	Hidden layer 64, plain SGD, batches of 64 words, 30 epochs, best epoch on validation; each kind of input gets its own learning rate from 0.25, 0.5, 1, 2 (chosen on validation, seed 0).

Why this test

A fair comparison needs each input at its own best learning rate and enough epochs; a first try with one shared rate and 10 epochs, during development, had every model still improving at its last epoch.

IMPLEMENTATION

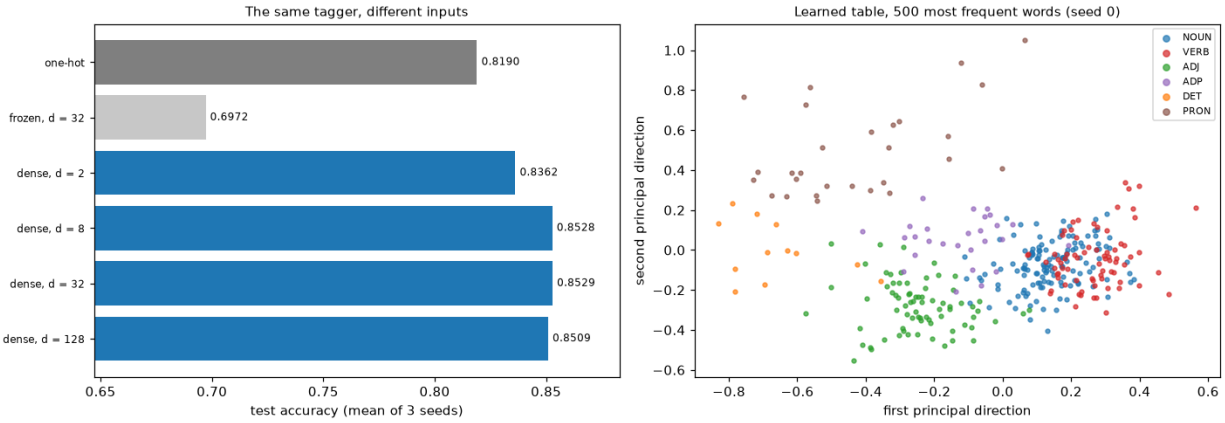
```
def inputs(self, win):
    if self.kind == "onehot":
        X = np.zeros((len(win), 3 * self.V))
        X[np.arange(len(win))[:, None], win + np.arange(3) * self.V] = 1.0
        return X
    return self.E[win].reshape(len(win), -1)
```

ACTUAL OUTPUT

```
validation accuracy, seed 0:    lr 0.25    lr 0.5    lr 1.0    lr 2.0
  onehot                        0.7804    0.8077    0.8185    0.8169   -> lr 1.0
  frozen                        0.6212    0.6717    0.6907    0.6907   -> lr 1.0
  dense                         0.8533    0.8496    0.8503    0.8428   -> lr 0.25
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
input             weights   seed 0   seed 1   seed 2     mean  best epoch  s / epoch
one-hot            400913   0.8190   0.8241   0.8140   0.8190        24.3       4.70
frozen, d = 32       7313   0.6940   0.6975   0.7000   0.6972        28.0       0.15
dense, d = 2         5717   0.8345   0.8355   0.8386   0.8362        29.0       0.13
dense, d = 8        19361   0.8564   0.8497   0.8522   0.8528        18.3       0.16
dense, d = 32       73937   0.8517   0.8527   0.8543   0.8529        22.3       0.27
dense, d = 128     292241   0.8541   0.8470   0.8517   0.8509        17.3       1.30
(weights: the ones training changes; the frozen table is not counted. best epoch: mean of the
    seeds)
Actual output from this lesson's run.
```



Left: test accuracy by input, mean of three seeds. Right: the learned table (d = 32) of the 500 most frequent words, projected on its two principal directions, coloured by each word's most common tag (Day 87 covers this projection properly).

What it means

Dense beats one-hot: 0.8528 with d = 8 against 0.8190, with 19361 trained weights against 400913, and about 4.70 s against 0.16 s per epoch. A likely reason, not tested here: one shared table gets updated by a word at every position it appears in, while the one-hot model learns separate weights for the word on the left, in the middle and on the right.

Learning the table is what matters: frozen random vectors reach only 0.6972. **Size saturates early**: d = 8, 32 and 128 all give about 0.85; even 2 numbers per word give 0.8362. The best dense learning rate, 0.25, was the smallest tried, so a smaller one might do slightly better.

WHY CHOOSE THIS?

Small, fast, and learns what the task needs.

WHY NOT THIS? (TRADEOFFS)

The vectors mean only what this task taught them (tags here).

4. What the learned table knows

DEFINITION — start here, no prior knowledge assumed

Cosine similarity between rows of the learned table: the five nearest words to a few words, and, for the 500 most frequent words, how often a word's nearest neighbour has the same most common tag.

REAL-WORLD SCENARIO

Books shelved by their few numbers: are the cookbooks next to each other?

MATHEMATICS

$$cos(u, v) = u \cdot v / (|u| \, |v|)$$

ACTUAL OUTPUT

```
  the      -> an, all, some, these, this
  is       -> was, are, 'm, does, could
  good     -> illegal, dead, great, nice, public
  said     -> reports, using, wants, moving, sounds
  you      -> we, it, they, i, something
  very     -> too, pretty, how, overall, always
  3        -> two, 2000, 2005, 2003, 8
  january  -> $, july, charge, india, president
nearest neighbour has the same most-common tag (500 most frequent words): learned 0.854, frozen
    random 0.136; two random words 0.138
Actual output from this lesson's run.
```

What it means

Grammatical neighbours: 'is' sits next to 'was' and 'are', 'you' next to 'we' and 'they', 'the' next to 'an' and 'this'. Meaning is only partly there ('good' next to 'illegal' and 'dead' as well as 'great' and 'nice'): the table learned from tags, so words that take the same tag end up together. Measured, 0.854 of the 500 commonest words have a nearest neighbour sharing their tag; the random table gives 0.136, the same as two random words (0.138).

WHY CHOOSE THIS?

Similarity comes for free from training.

WHY NOT THIS? (TRADEOFFS)

It is similarity for this task; Day 86 learns vectors from raw text instead.

Interview preparation — technical questions

Q: What is a one-hot vector times a weight matrix?

A: The matrix's row for that word: an embedding lookup (0.0e+00 difference here).

Q: How is an embedding table's gradient computed?

A: Scatter-add: each position's gradient is added to its word's row (np.add.at).

Q: How far apart are two different one-hot vectors?

A: Always sqrt 2, with dot product 0.

Q: What did the tagger score with each input?

A: One-hot 0.8190; dense d = 8 0.8528; frozen random 0.6972.

Q: How many numbers per word were enough?

A: About 8: d = 8, 32 and 128 all gave about 0.85.

Interview preparation — theoretical questions

Q: Why are dense embeddings better than one-hot input?

A: Fewer weights, shared across positions, and similar words can get similar vectors.

Q: Why did the frozen table do badly?

A: Random vectors carry no information about words; the hidden layer must untangle arbitrary positions.

Q: What do embeddings learned from a task encode?

A: Whatever helps the task: here grammatical category (0.854 nearest-neighbour tag agreement).

Q: Why choose the learning rate per input?

A: So that one input is not judged at another's best setting.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> An embedding is a lookup: a one-hot vector times a table, done without the multiplication. Learned and shared across positions, a table of a few numbers per word beats one-hot input with far fewer weights, and its rows arrange themselves so that words used alike sit together.

- One-hot @ E equals E[ids] exactly; one-hot inputs 16.7 MB against 4.0 kB of ids.
- Test accuracy: one-hot 0.8190 (400913 weights); dense d = 8 0.8528 (19361); frozen 0.6972.
- Nearest neighbour shares the tag: learned 0.854, random 0.136, chance 0.138.

Suggested watch order, if starting from zero

- 1. What are Word Embeddings? (IBM Technology) — embeddings in a few minutes.
- 2. Word Embedding and Word2Vec, Clearly Explained!!! (StatQuest with Josh Starmer) — from one-hot to learned vectors, and Word2Vec.

Day 86 preview

- **Word2Vec's skip-gram, from scratch.** Today's table learned from tags. Day 86 learns word vectors from raw text alone, by predicting each word's neighbours.
- Today's exercise script, `day85_exercise.py`, is an embedding layer, forward and backward: the second part of Day 90's model.